APMA 4302: Methods in Computational Science

Spring 2026

Marc Spiegelman (APAM/DEES)

Lecture 17

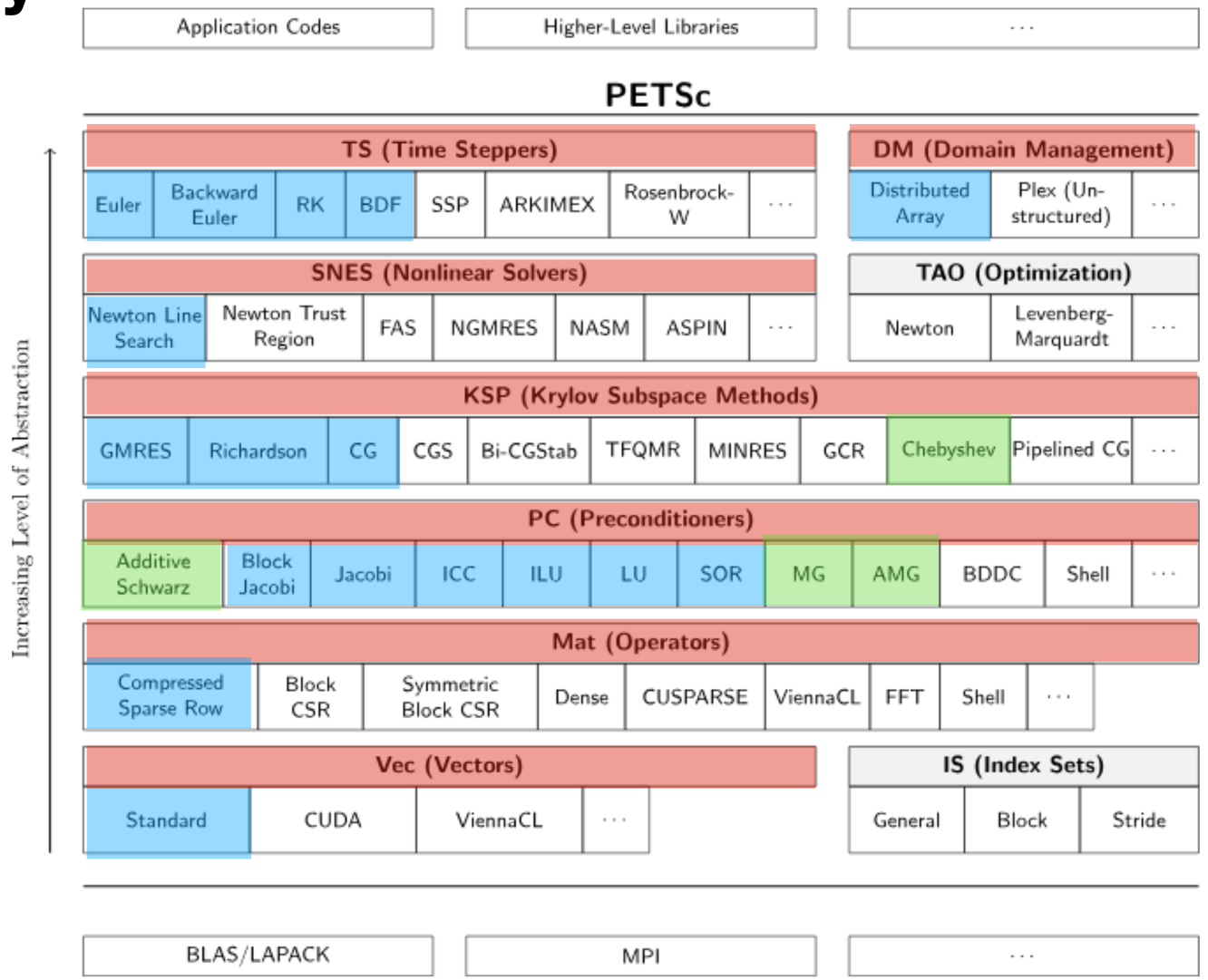
Preconditioners for PDE's (Beuler Ch 6)

Outline

- Where we are
- Quick Review of basic preconditioners
- Overview of advanced preconditioners
  - Parallel Preconditioners: Domain Decomposition (PETSc bjacobi and ASM)
  - Multi-grid Preconditioners (PETSc mg and gamg)
  - Physics based block-pre conditioners (PETSc field split)

Where we are

The PETSc object hierarchy



Review Preconditioning

Left and Right preconditioners

Given a Linear problem

$$A\mathbf{x} = \mathbf{b} \qquad \text{or} \qquad J(\mathbf{x})\delta = -\mathbf{F}(x)$$

and an invertible matrix M, equivalent problems can be solved using

Left Preconditioning: $M^{-1}A\mathbf{x} = M^{-1}\mathbf{b}$

Right Preconditioning: $AM^{-1}\mathbf{y} = \mathbf{b}, \quad M\mathbf{x} = \mathbf{y}$

Where M is “easy to invert” and $\kappa(M^{-1}A)$ or $\kappa(AM^{-1}) \approx 1$

Chosen well, a good preconditioner can significantly improve the convergence of iterative schemes

Review: Preconditioned Richardson Iteration

-ksp_type richardson -pc_type X

Preconditioned Richardson Iteration:

$$\mathbf{u}_{k+1} = \mathbf{u}_k + M^{-1}\mathbf{r}_k$$

or given, $\mathbf{u}_{true} = \mathbf{u}_k + \mathbf{e}_k$, and $A\mathbf{e}_k = \mathbf{r}_k$, equivalently

$$\mathbf{e}_{k+1} = \left(I - M^{-1}A\right)\mathbf{e}_k$$

Convergence requires that $\rho\left(I - M^{-1}A\right) < 1$

Review: Preconditioned Richardson Iteration

-ksp_type richardson -pc_type X

Some simple splitting preconditioners (assuming $A = D + L + U$)

Richardson Iteration: $M = I$: -pc_type none

Modified Richardson: $M = \frac{1}{\omega} I$: -ksp_richardson_scale omega

Jacobi: $M = D$: -pc_type jacobi

Damped Jacobi: $M = \frac{1}{\omega} D$: (combine both)

Gauss-Seidel: $M = \frac{1}{\omega} D + L$: -pc_type sor_forward

$M = \frac{1}{\omega} D + U$: -pc_type sor_backwards

Symmetric SOR: $M = \frac{\omega}{2-\omega} \left(\frac{1}{\omega} D + L \right) D^{-1} \left(\frac{1}{\omega} D + U \right)$: -pc_type sor

These are lousy solvers for poisson like problems: but good “smoothers”

Review: Preconditioned Krylov schemes

-ksp_type X (cg, gmres, chebyshev,…) -pc_type X

Stronger preconditioners

Direct solvers $(M = A)$	: -pc_type lu
	: -pc_type cholesky
Incomplete Factorizations	: -pc_type ilu
	: -pc_type icc
+ more exotic pc's	

These are decent solvers, but do not scale and are not optimal

Convergence Behavior for 2-D poisson

./fish -ksp_type cg -pc_type icc -ksp_rtol 1.e-10 -ksp_atol 1.e-10

$$\nabla^2 u = f$$

on , $\Omega = [0,1] \times [0,1]$

with dirichlet BC's with
$N \times N$ unknowns

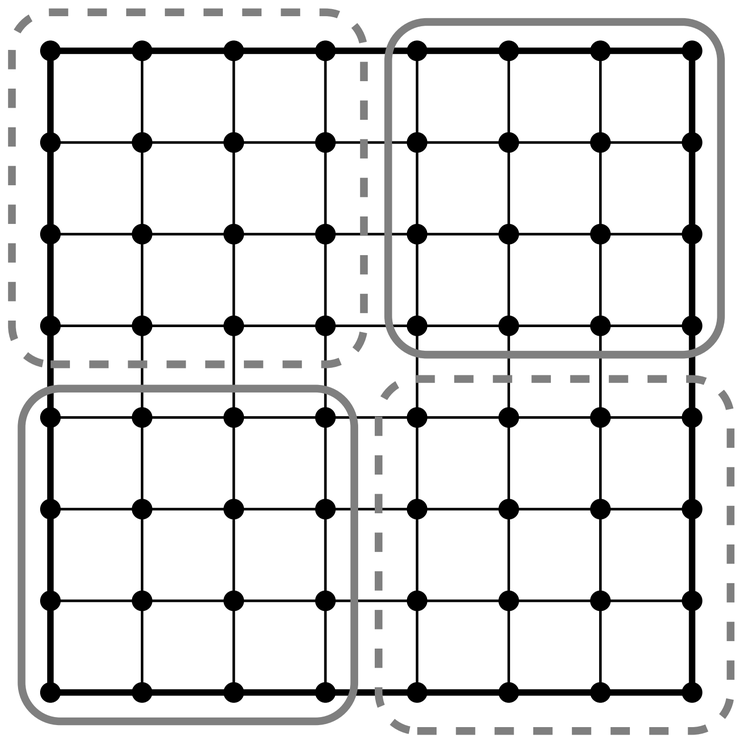
Single core

N	Iterations	‖r‖	‖u-u_true‖_h	KSPSolve (s)
5	7	2.730E-20	1.745E-04	4.4E-05
9	13	4.051E-11	4.586E-05	6.9E-05
17	21	5.7194E-10	1.160E-05	1.8E-04
33	39	8.6023E-10	2.907E-06	8.12E-04
65	76	7.7386E-10	7.271E-07	4.63E-03
129	145	1.6928E-09	1.818E-07	3.08E-02
257	278	2.3851E-09	4.546E+00	2.16E-01

Can we do better?

Parallel Domain Decomposition pre-conditioners

-ksp_type cg -pc_type bjacobi -sub_ksp_type none -sub_pc_type cholesky



from Beuler Fig 6.8

N	Iterations	‖r‖	‖u-u_true‖_h	KSPSolve (s)
33	27	2.096E-09	2.907E-06	1.169E-03
65	38	2.960E-09	7.271E-07	1.64E-03
129	53	6.39E-09	1.818E-07	6.44E-03
257	74	1.558E-08	4.546E-08	4.34E-02
513	103	3.6E-08	1.137E-08	2.70E-01

Parallel Domain Decomposition pre-conditioners

Additive Schwarz Method -pc_type asm

-ksp_type cg -pc_type asm -pc_asm_overlap X

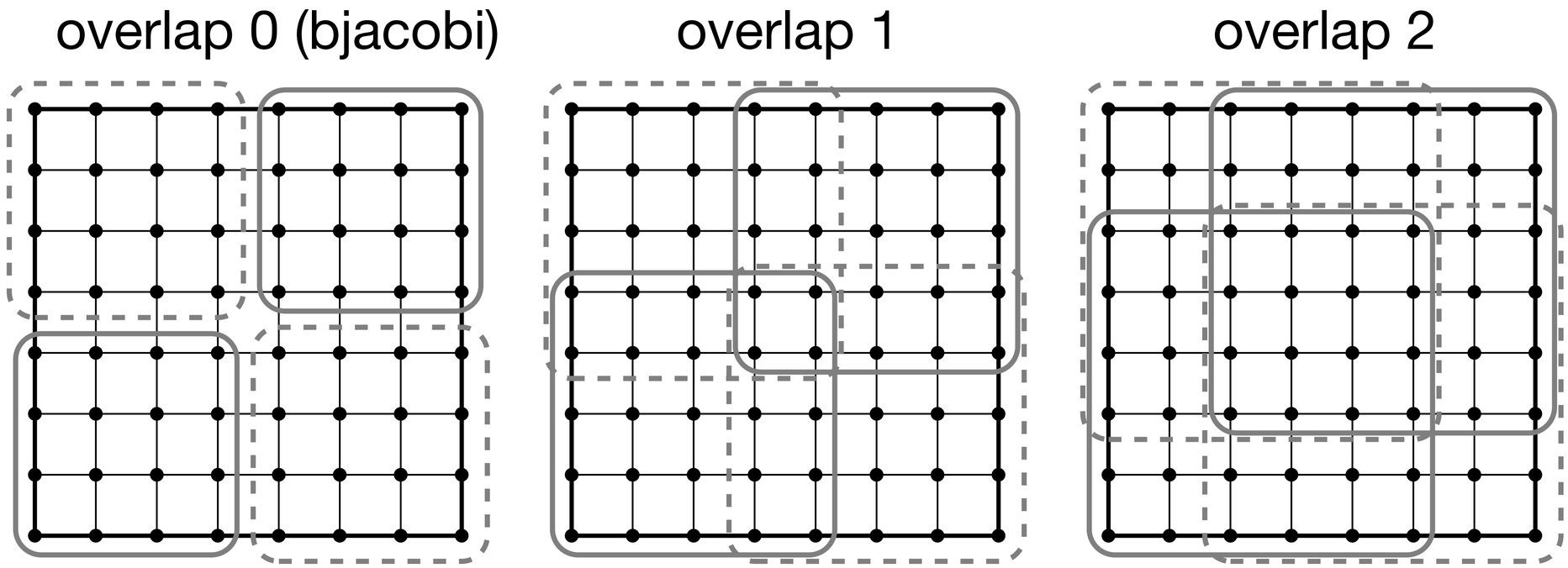


Figure 6.8. *Subgrid decompositions of a* 8×8 *grid into* $p = 4$ *equal-sized subgrids with overlaps of* 0 *(left),* 1 *(middle), and* 2 *(right) grid points.*

Parallel Domain Decomposition pre-conditioners

Additive Schwarz Method -pc_type asm

```
$ mpiexec -n P ./fish -fsh_dim D -da_refine L -ksp_converged_reason \
    -pc_type asm -sub_pc_type cholesky -pc_asm_overlap X
```

Table 6.1. *Number of CG iterations from ASM runs of* `fish.c` *with* `P` *processes, equal to the number of subdomains, and varying overlaps* `X`*. Grids are* $m \times m$ *in 2D and* $m \times m \times m$ *in 3D.*

		Overlap `X`			
	m	0	1	2	4
2D	17	13	11	11	12
	33	18	13	11	10
(P $= 4$)	65	25	18	15	12
	129	33	21	19	15
	257	44	28	23	20
3D	17	16	11	10	9
	33	21	15	12	10
(P $= 8$)	65	29	20	16	12

Can we do better?

towards Geometric Multi-grid: issues

To understand why multigrid is an optimal solver, it's worth understanding why simple preconditioners are so poor. Actually two reasons

- Physics: 2nd order elliptic PDE's are dominated by long-range interactions

- Numerics: Simple iterative schemes are dominated by local interactions

Physics of Poisson Equation

Poisson's Equation

$$-\nabla^2 u = f$$

is the steady state solution of

$$\frac{\partial u}{\partial t} = \nabla^2 u + f$$

and is dominated by long-wavelength interactions

Physics of Poisson Equation

But simple preconditioners act locally

Can actually show that Damped Jacobi preconditioned richardson, is equivalent to stable Euler's method. Which is controlled by local, high-frequency components.

Damped Jacobi: $\mathbf{u}_{k+1} = \mathbf{u}_k + \omega D^{-1}\mathbf{r}_k$

Euler's method: $\mathbf{u}_{k+1} = \mathbf{u}_k + \Delta t \mathbf{r}_k$

where: $\mathbf{r}_k = \mathbf{f} - A\mathbf{u}_k, \quad A = \frac{1}{h^2}\text{tridag}\begin{bmatrix}-1 & 2 & -1\end{bmatrix},$

and $\lambda_j(A) = \frac{2}{h^2}\left[1 - \cos(j\pi h)\right]$

Damped Jacobi as a smoother

Damped Jacobi: $\mathbf{u}_{k+1} = \mathbf{u}_k + \omega D^{-1}\mathbf{r}_k$

or in correction form: $\mathbf{e}_{k+1} = \left[I - \omega D^{-1}A\right]\mathbf{e}_k$

or given: $A = \frac{1}{h^2}\text{tridag}\left[-1 \;\; 2 \;\; -1\right],$

can write in stencil form to show that

$$\mathbf{e}_{k+1} = \left[\frac{\omega}{2} \quad (1-\omega) \quad \frac{\omega}{2}\right]\mathbf{e}_k$$

Damped Jacobi suppresses eigenmodes unevenly

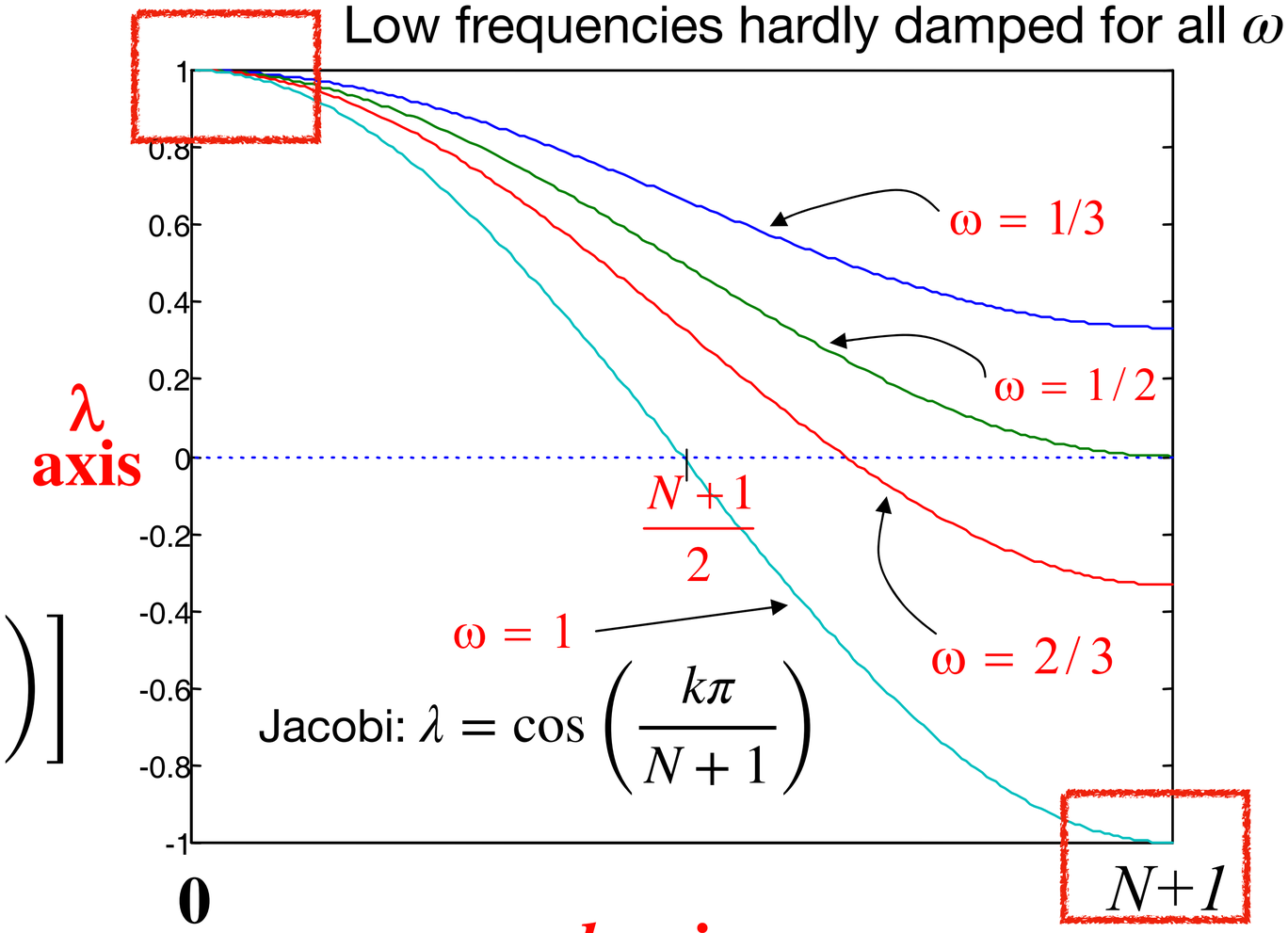
From David Keyes notes on Multigrid

Given iteration Matrix

$$B = \left[I - \omega \frac{h^2}{2} A \right]$$

$$\lambda(B) = 1 - \omega \frac{h^2}{2} \lambda_A$$

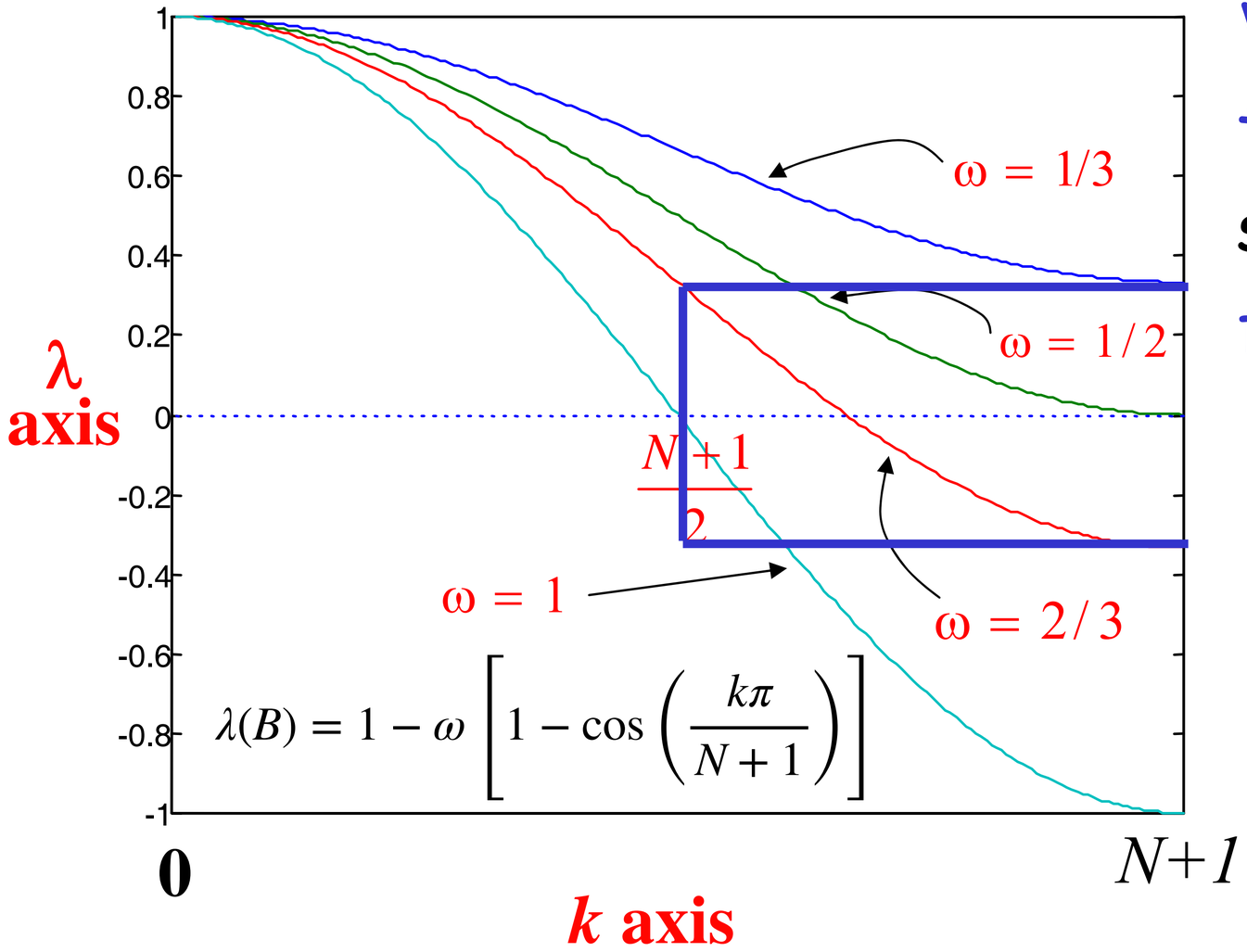
$$\lambda(B) = 1 - \omega \left[1 - \cos\left(\frac{k\pi}{N+1} \right) \right]$$



For Jacobi, neither are high k

Optimal smoothers

From David Keyes notes on Multigrid



What value of ω gives the best damping of short waves or high frequencies $N/2 \le k \le N$?

Choose ω such that

$$\lambda_{N/2}(B) = -\lambda_N(B)$$

$$\Rightarrow \omega \approx \frac{2}{3}$$

Effect of smoothers on random vectors

From Beuler

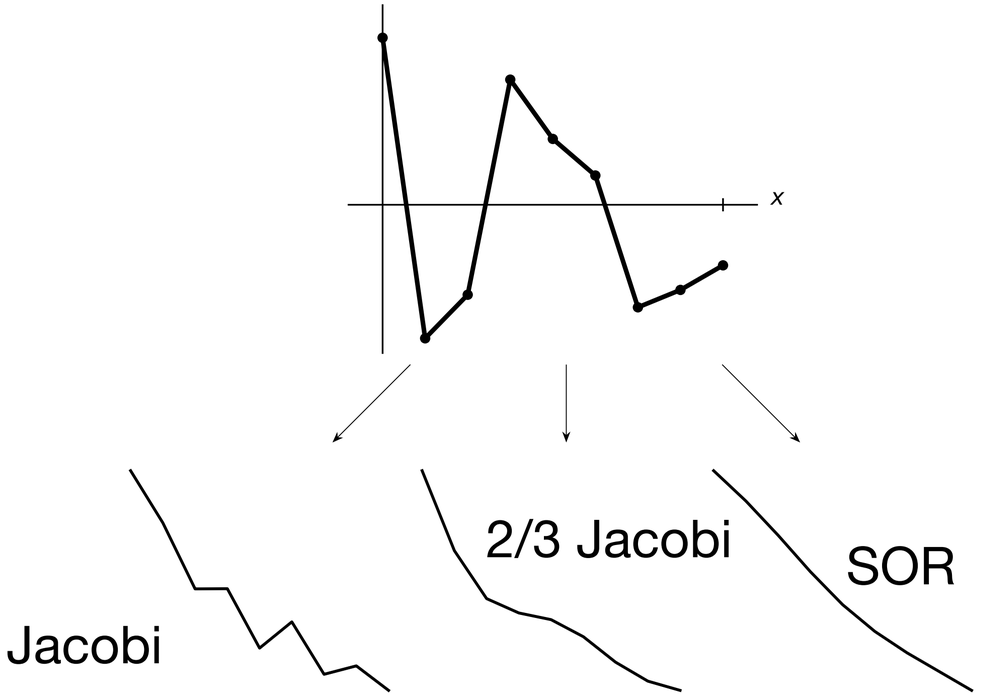


Figure 6.2. *Smoothing of a random 1D function (top) on a 9-point grid using 4 iterations of Jacobi (left), $\alpha = 2/3$ weighted Jacobi (middle), and GS (right) iterations, respectively, on a discrete Laplacian matrix.*

Spatial domain

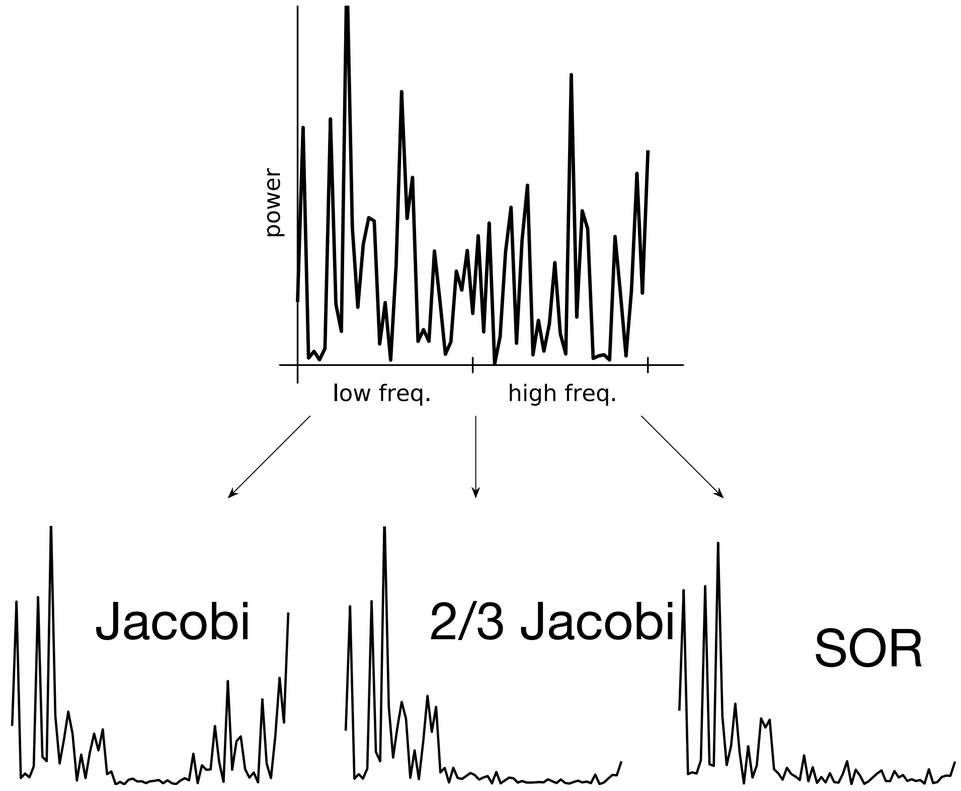


Figure 6.3. *The power spectrum of a random initial vector (top), and from a single iteration, of Jacobi (left), $\alpha = 2/3$ weighted Jacobi (middle), and GS (right), on a 129-point grid.*

Frequency domain

Geometric Multigrid

The Big ideas

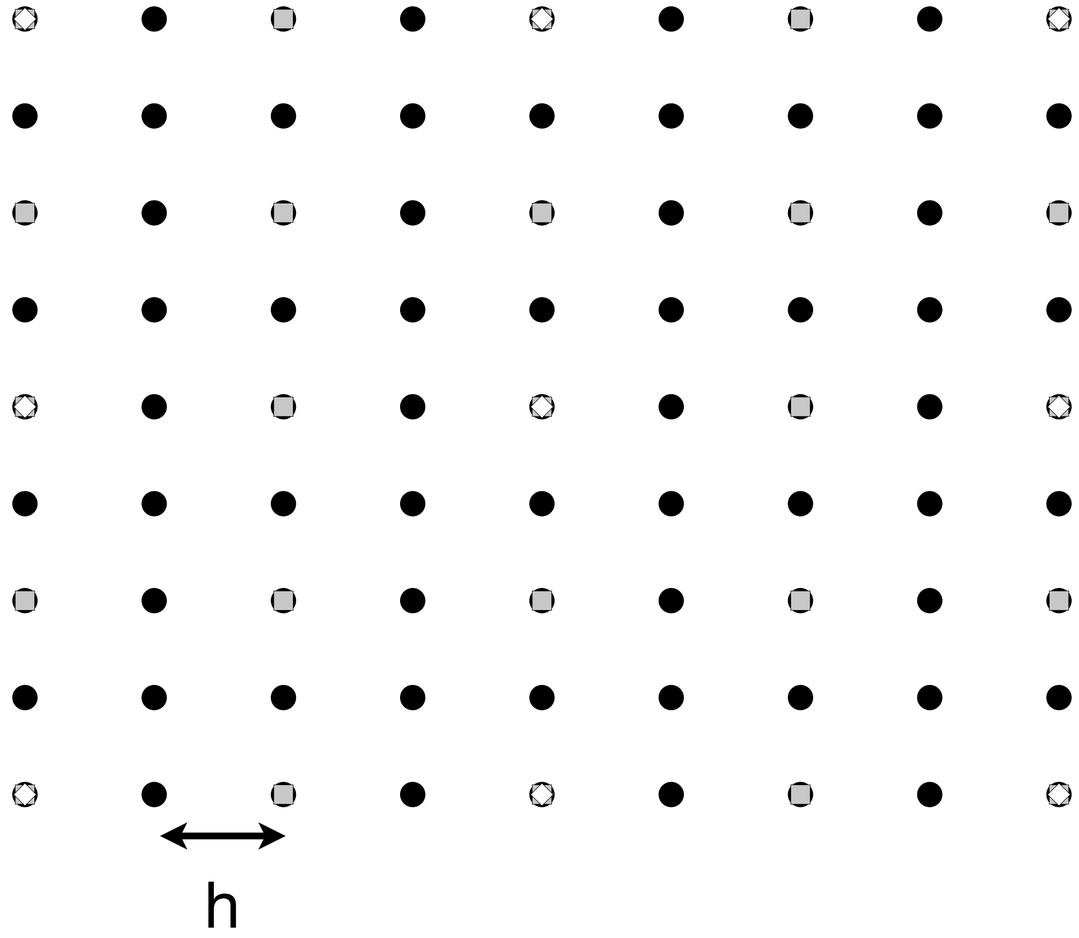
- Simple schemes like damped jacobi preconditioned richardson are bad solvers, but good *smoothers,* for a sub-set of frequencies
- I.e. they're really low-pass filters
- However, the frequencies are only defined by the mesh

$$\mathbf{e}_{k+1} = \begin{bmatrix} \frac{\omega}{2} & (1-\omega) & \frac{\omega}{2} \end{bmatrix} \mathbf{e}_k$$

- And what is smooth on a fine mesh is rough on a coarse mesh

Multigrid: Nested grids

3x3 coarse mesh with two levels of refinement



A 3-level nested grid

- level 0, grid spacing h
- level 1 grid, spacing 2h
- level 2, grid spacing 4h

Introduction to Geometric Multigrid

Outline

- Ingredients for Geometric Multigrid
- Basic 2-level scheme
- V-cycles and Full Multigrid
  - Computational Complexity
- Parallel Multigrid issues
- PETSc implementation
  - Options
  - Performance Comparisons
  - Algebraic Multigrid (gamg, hypre, ml)

Multigrid is really a recipe…

Basic Ingredients

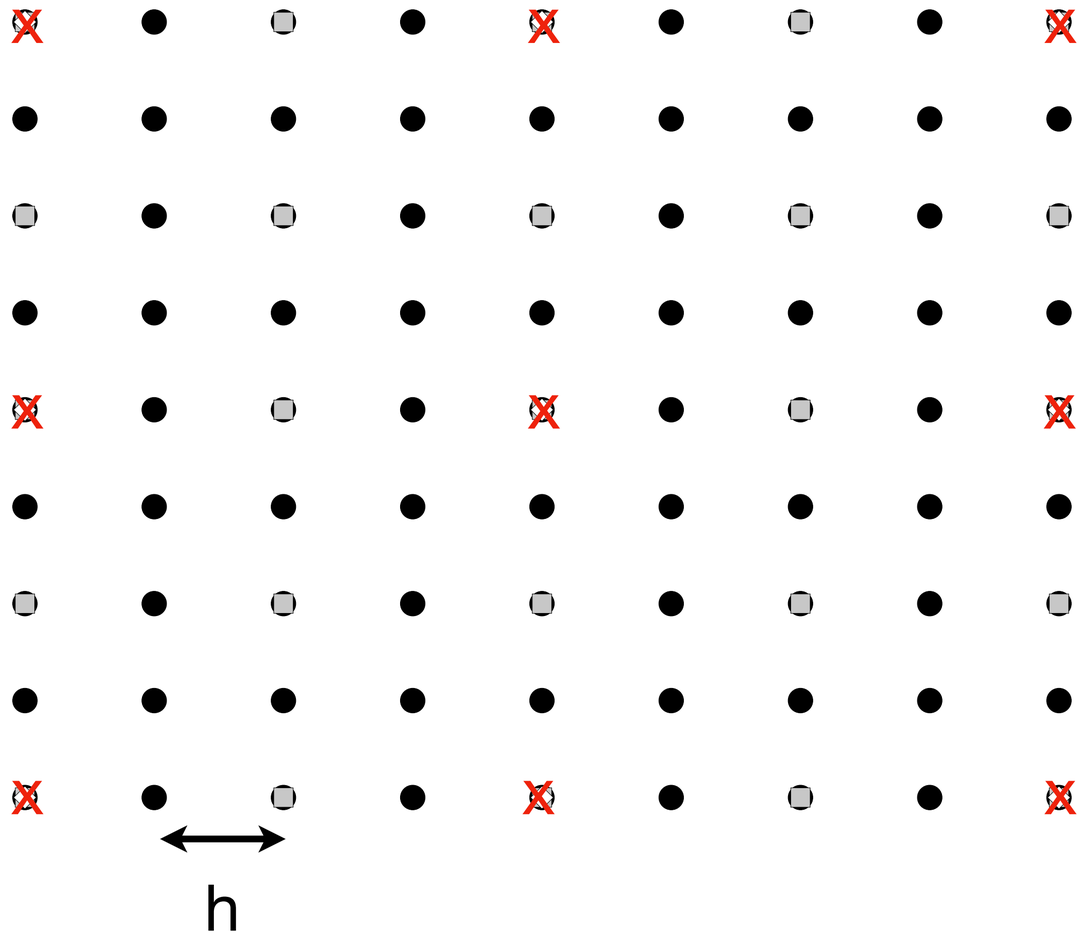
- Multi-level nested mesh
- Grid Transfer Operators
  - Fine to Coarse (Restriction): R_h^{2h}
  - Coarse to Fine (Prolongation/Interpolation): I_{2h}^h
- Grid Level operators: $A_h, A_{2h}, \ldots$
- Level smoothers (or solvers): (level wise ksp/pc pairs)

Smoothers: a review

- Classical Schemes: Preconditioned Richardson iteration
  - Gauss-Seidel (-ksp_type richardson -pc_type SOR)
  - damped Jacobi (-ksp_type richardson -ksp_richardson_scale 2/3 (1d), -pc_type jacobi
  - Don’t use undamped jacobi!
- More robust smoothers:
  - PETSc default: -ksp_type chebyshev -pc_type SOR
  - can use other -pc_types as well (icc, ilu)
- Point: not solvers, but effectively damp selected higher-frequency components

Multigrid: Nested grids

3x3 coarse mesh with two levels of refinement

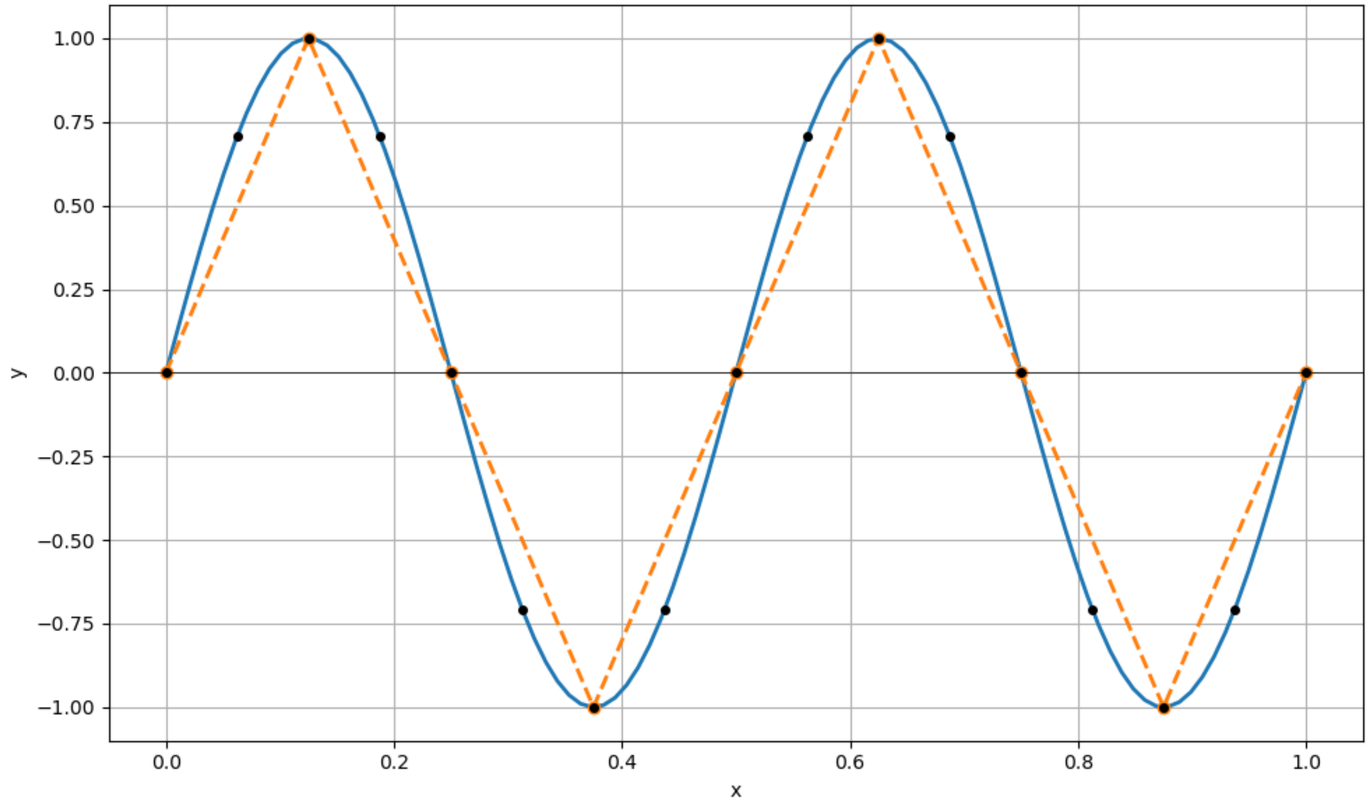


A 3-level nested grid

- level 2, grid spacing h
- level 1 grid, spacing 2h
- level 0, grid spacing 4h “coarse mesh”

Point: a smooth function on a fine grid, can appear rough on a coarse grid

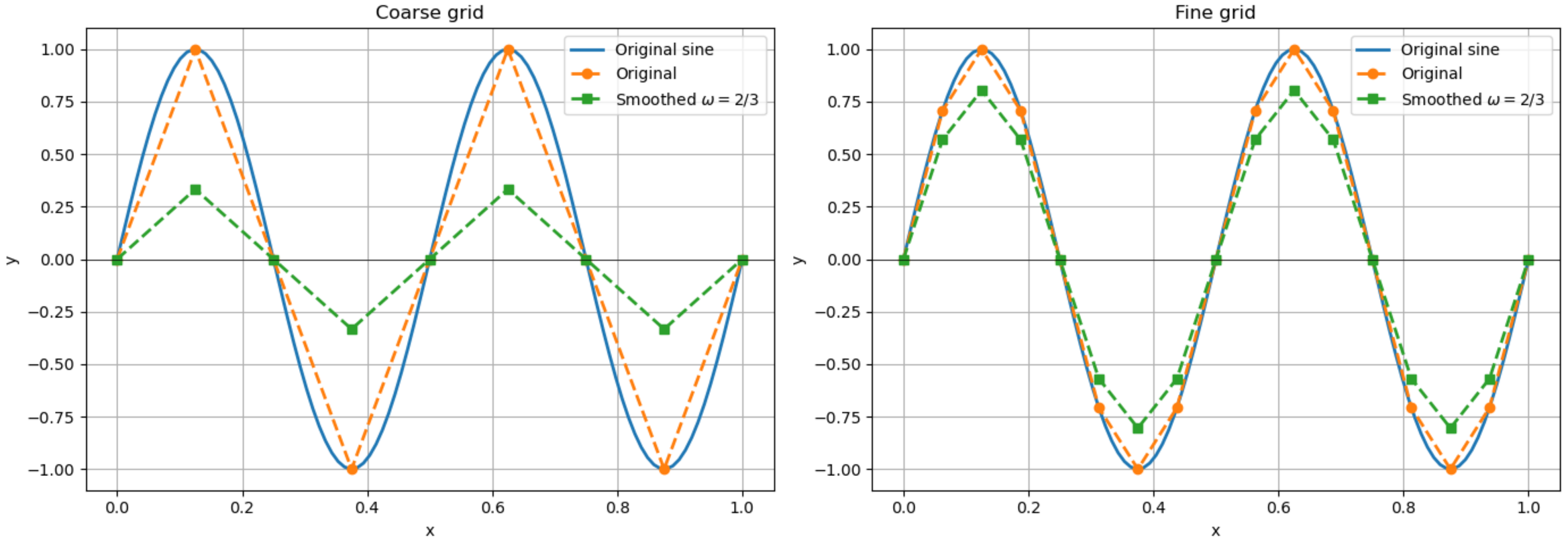
Multigrid: Nested grids



Point: a smooth function on a fine grid, can appear rough on a coarse grid

Multigrid: Nested grids

Application of damped Jacob ($\omega = 2/3$) to different resolution meshes



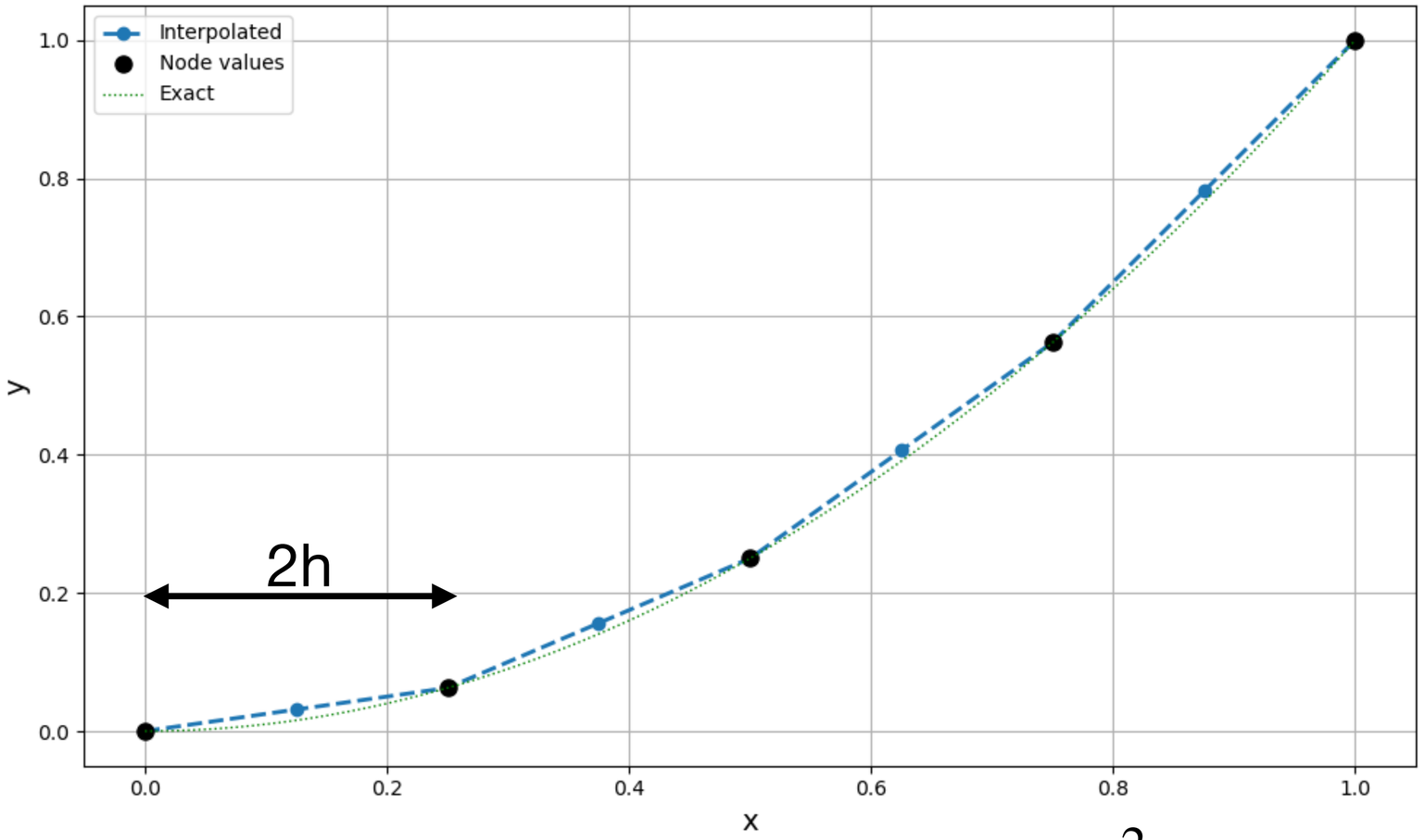
$$\begin{bmatrix} \frac{\omega}{2} & (1-\omega) & \frac{\omega}{2} \end{bmatrix} = \begin{bmatrix} \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \end{bmatrix}$$

The idea of Multigrid

- Use smoothers on a hierarchy of meshes to filter out mesh scale errors on each level
- But this requires being able to move information between mesh levels

Grid transfer operators

Coarse to fine (interpolation) I_{2h}^{h} or P_{int}



Interpolation of a smooth function x^2

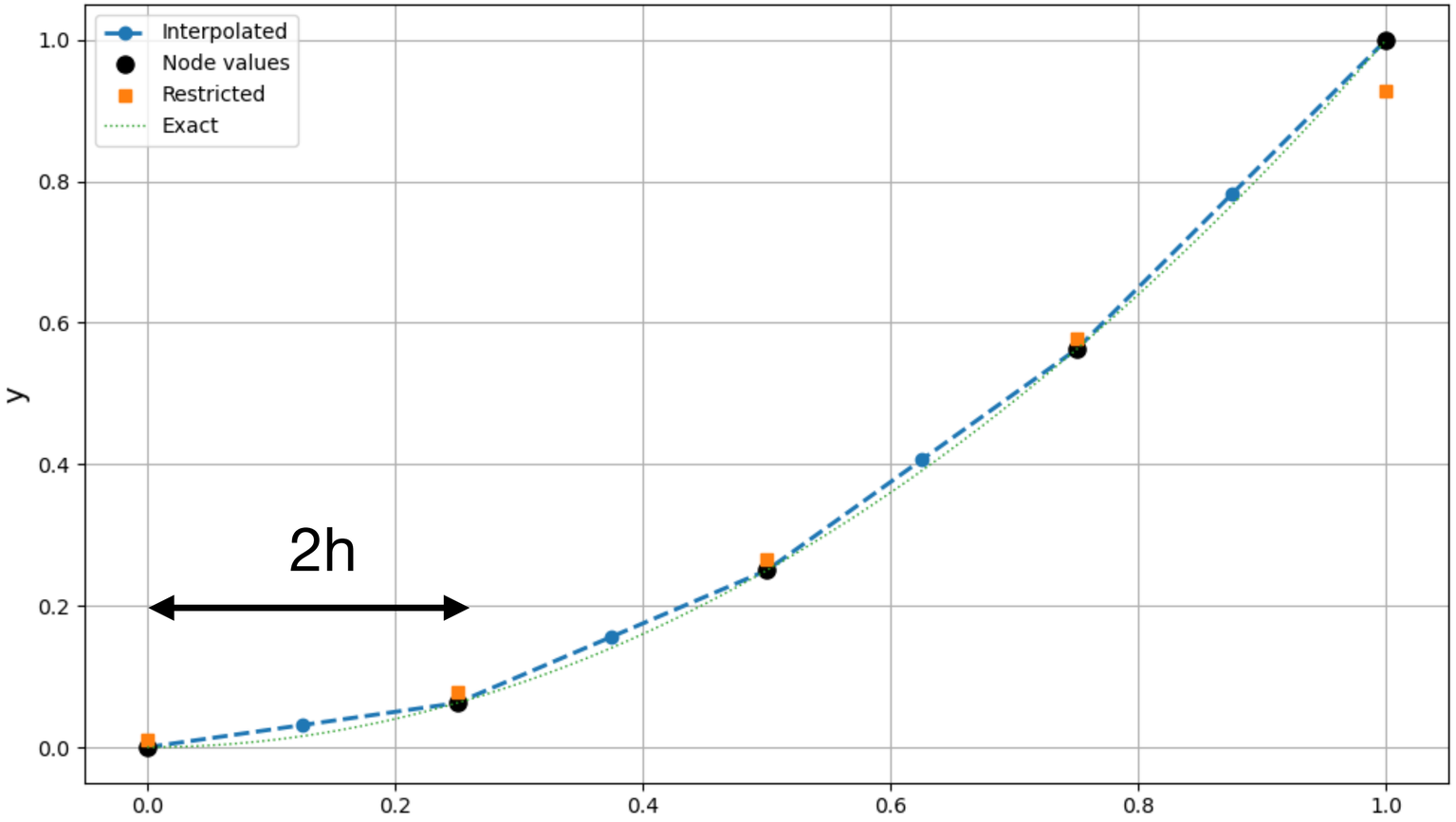
$$P_{\text{int}} = \begin{matrix} \\ \\ \\ \\ \\ \\ \\ \\ 9 \end{matrix} \overset{5}{\begin{bmatrix} 1 & & & & \\ 1/2 & 1/2 & & & \\ & 1 & & & \\ & 1/2 & 1/2 & & \\ & & 1 & & \\ & & 1/2 & 1/2 & \\ & & & 1 & \\ & & & 1/2 & 1/2 \\ & & & & 1 \end{bmatrix}}.$$

9x5 Linear Interpolation matrix

$$\mathbf{y}_h = P_{int}\mathbf{y}_{2h}$$

Grid transfer operators

Fine to coarse (restriction) R_h^{2h} or $\mathrm{diag}(\mathbf{c})P_{int}^T$ (full weighted)



Restriction of an interpolated function

$$\mathbf{y}_{2h} = R_h^{2h} I_{2h}^h \mathbf{y}_h$$

$$R_{\mathrm{fw}} = \begin{bmatrix} 2/3 & 1/3 & & & & & & & \\ & 1/4 & 1/2 & 1/4 & & & & & \\ & & & 1/4 & 1/2 & 1/4 & & & \\ & & & & & 1/4 & 1/2 & 1/4 & \\ & & & & & & & 1/3 & 2/3 \end{bmatrix}$$

9

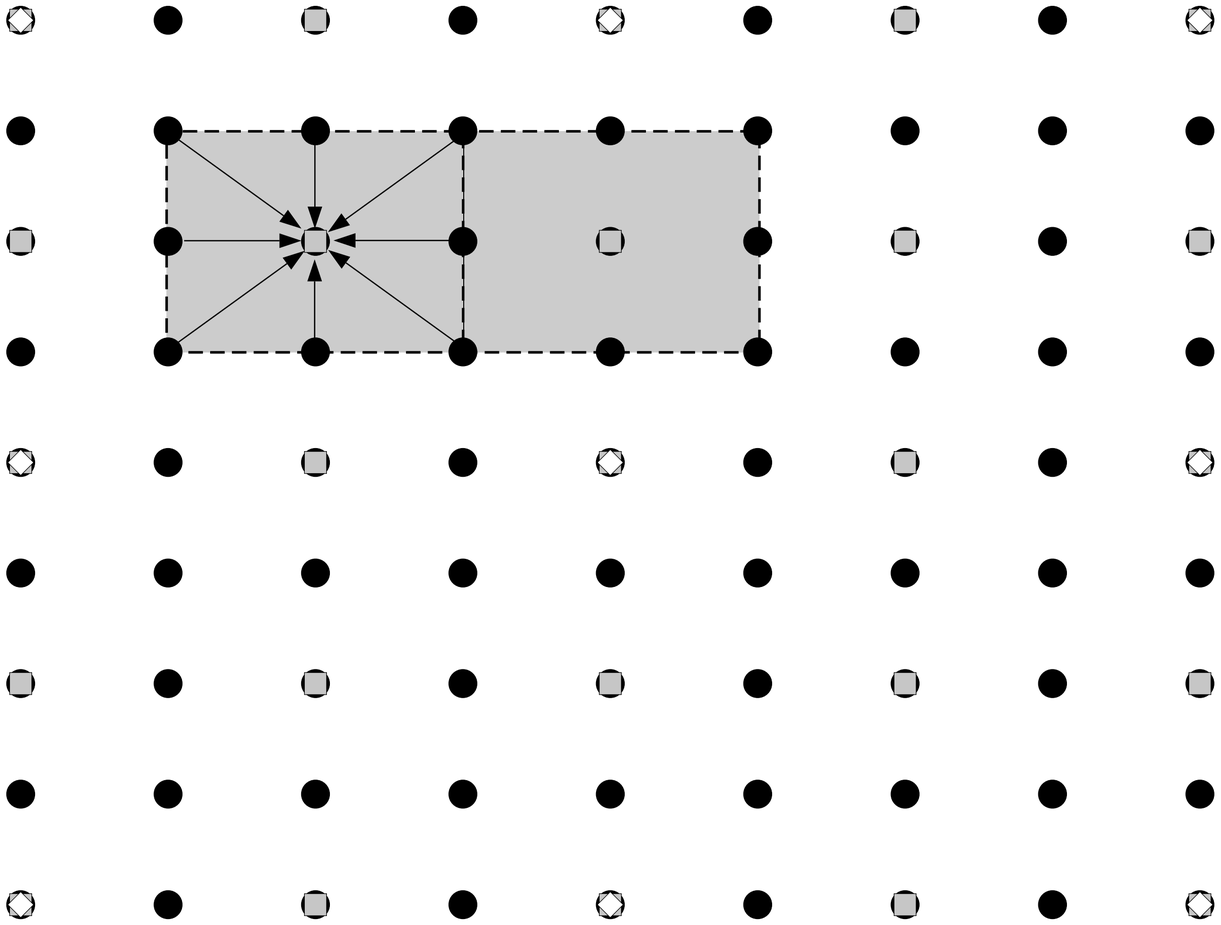
5

5x9 Linear Interpolation matrix

$$\mathbf{y}_{2h} = R_h^{2h} \mathbf{y}_h$$

Full-weighted restriction in 2-D

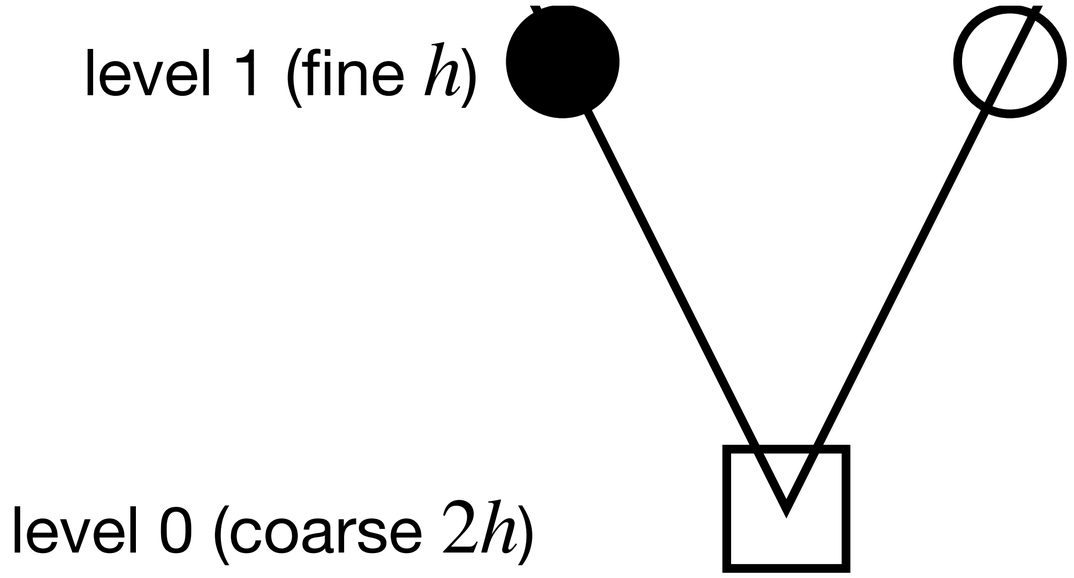
Fine to Coarse (restriction) R_h^{2h}



2-level correction scheme

pseudo-code

function TWOGRID$(A, \mathbf{b}, \mathbf{w})$
  $\mathbf{v} \leftarrow \mathbf{w}$
  for $k = 1, 2, \ldots, \nu_1$ — *pre-smoothing ν_1 times*
    $\mathbf{v} \leftarrow \mathcal{S}_1(A, \mathbf{b}, \mathbf{v})$
  $\mathbf{r}^{2h} \leftarrow P_{\text{int}}^{\top}(\mathbf{b} - A\mathbf{v})$ — *coarse-grid correction*
  solve $A^{2h}\mathbf{z}^{2h} = \mathbf{r}^{2h}$ (6.24)
  $\mathbf{v} \leftarrow \mathbf{v} + P_{\text{int}}\mathbf{z}^{2h}$
  for $k = 1, 2, \ldots, \nu_2$
    $\mathbf{v} \leftarrow \mathcal{S}_2(A, \mathbf{b}, \mathbf{v})$ — *post-smoothing ν_2 times*
  return $\mathbf{v}$



2-level correction scheme

on the board

Coarse grid operators A^{2h}

two ways to calculate

- Rediscretization: (PETSc default)
  - A^{2h}: just re-assemble the linear operator A but on the coarser mesh
- Galerkin coarsening (-pc_mg_galerkin_)
  - set $A^{2h} = P_{int}^T A_h P_{int}$

$$A^{2h} = \begin{bmatrix} 8 & & & & \\ & 8 & -4 & & \\ & -4 & 8 & -4 & \\ & & -4 & 8 & \\ & & & & 8 \end{bmatrix}.$$

•Rediscretization

$$\hat{A}^{2h} = \begin{bmatrix} 20 & & & & \\ & 8 & -4 & & \\ & -4 & 8 & -4 & \\ & & -4 & 8 & \\ & & & & 20 \end{bmatrix}.$$

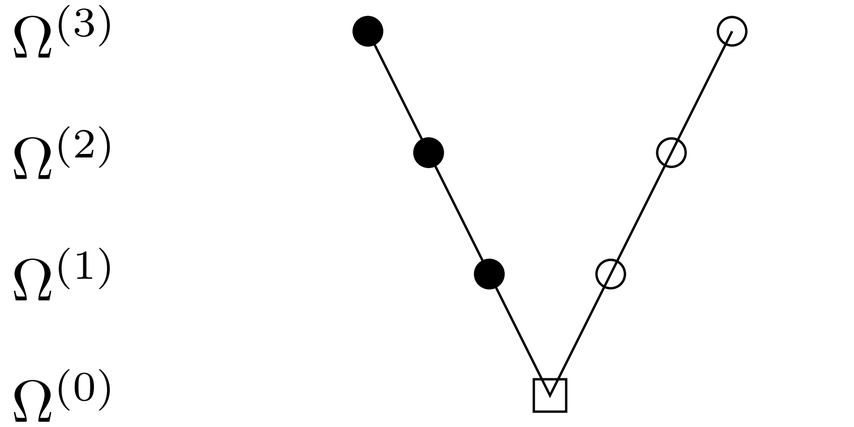
•Galerkin

from 2 levels to N levels

the V-cycle (recursive form)

function VCYCLE($A, \mathbf{b}, \mathbf{w}, l$)
  if $l == 0$
    solve $A\mathbf{v} = \mathbf{b}$, e.g., by a direct solver
  else
    $\mathbf{v} \leftarrow \mathcal{S}_1^{\nu_1}(\mathbf{w})$ smooth ν_1 times
    $\mathbf{r}^C \leftarrow P_{\text{int}}^{\top}(\mathbf{b} - A\mathbf{v})$ restrict the residual
    form A^C coarser operator
    $\mathbf{z}^C \leftarrow \text{VCYCLE}(A^C, \mathbf{r}^C, 0, l-1)$ calculate coarse correction (recursive)
    $\mathbf{v} \leftarrow \mathbf{v} + P_{\text{int}}\mathbf{z}^C$ interpolate correction and add
    $\mathbf{v} \leftarrow \mathcal{S}_2^{\nu_2}(\mathbf{v})$ smooth ν_2 times
  return $\mathbf{v}$

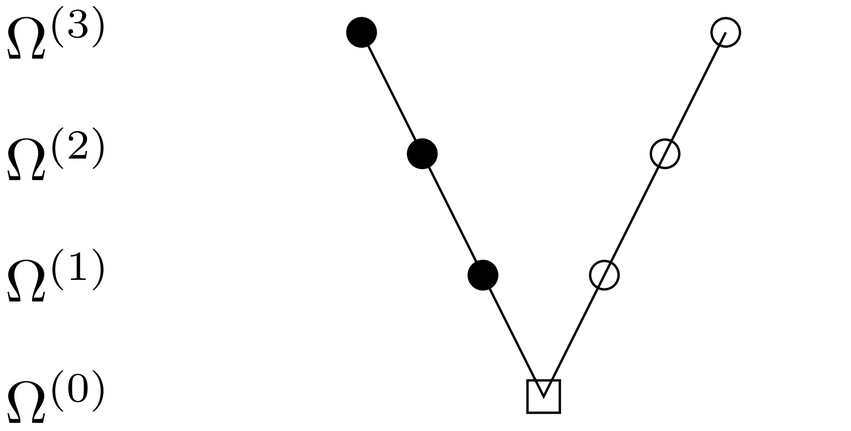
4-level V-cycle



`-pc_mg_cycle_type v`

from 2 levels to N levels

the V-cycle (recursive form)



`-pc_mg_cycle_type v`

Meshes in 2-D 4 level scheme

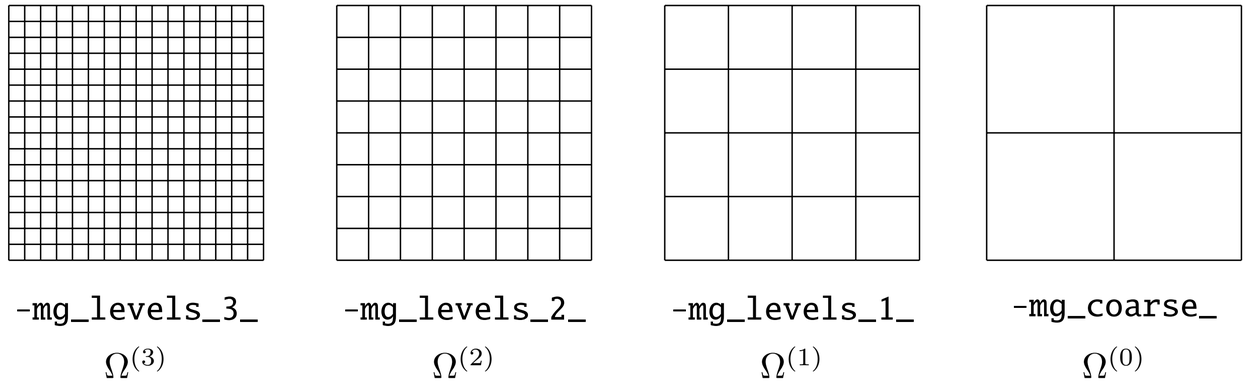


Figure 6.12. *The 2D grids used in* `./fish -da_refine 3 -pc_type mg`.

Computational Work in a V-cycle

Dominated by work on the fine mesh

In 2-D:

Meshes in 2-D 4 level scheme

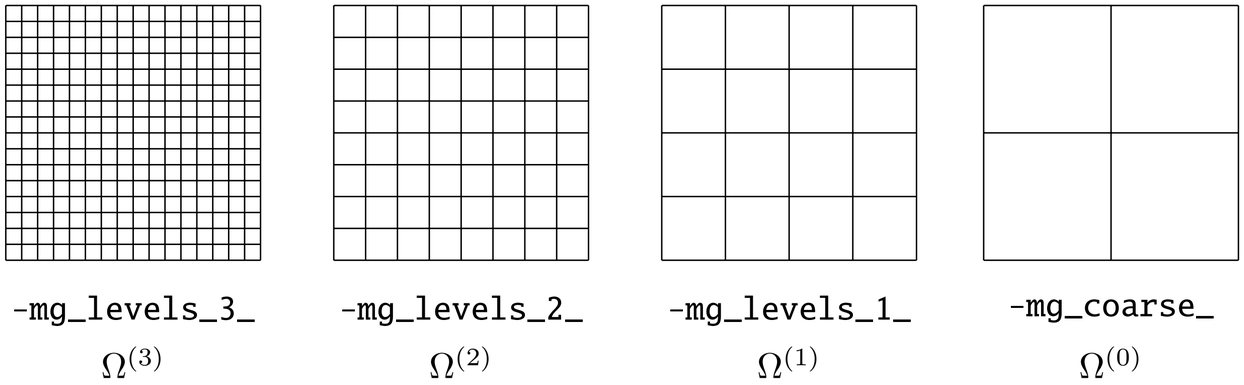
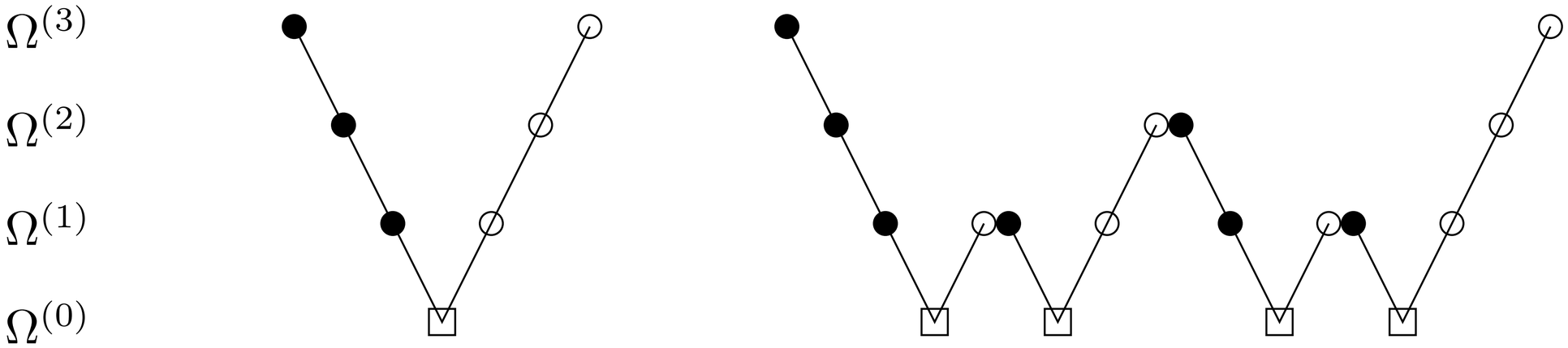


Figure 6.12. *The 2D grids used in* `./fish -da_refine 3 -pc_type mg`.

Other multi-grid cycles

V-cycle and W-cycles



`-pc_mg_cycle_type v` `-pc_mg_cycle_type w`

4-level V-cycle and W-cycle (calls recursive part twice)

Other multi-grid cycles

the V-cycle (recursive form)

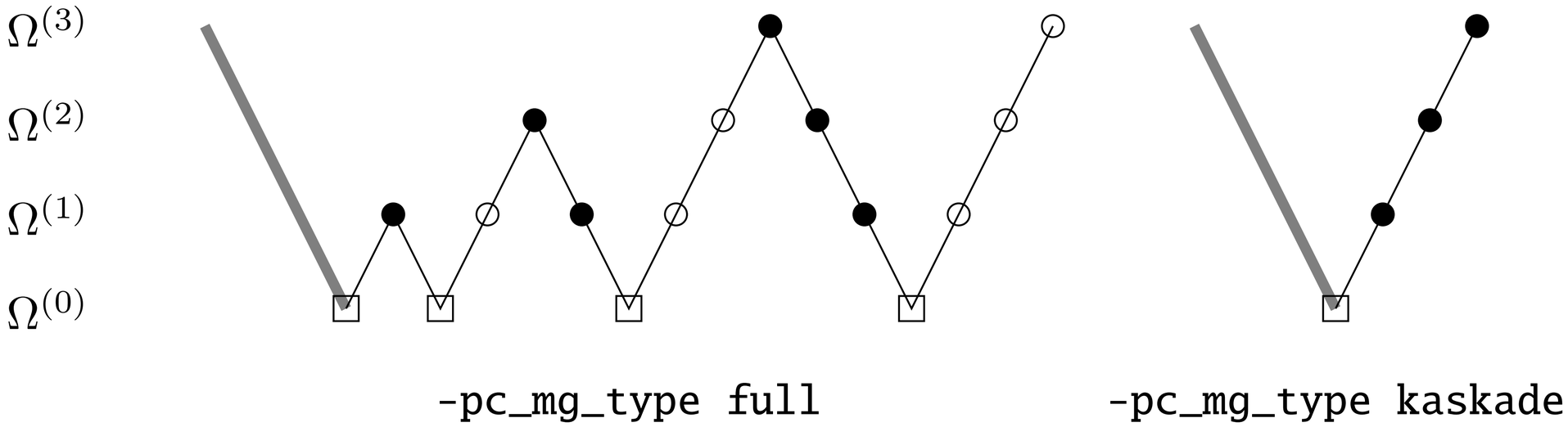


Figure 6.14. *One full (left) or Kaskade (right) cycle for a* `-da_refine 3 -pc_type mg` *run. The initial coarsening phase only restricts the residuals.*

FMG and Kaskade start at the Coarse level as an initial guess and refines

Other multi-grid cycles

Full Multigrid and Kaskade

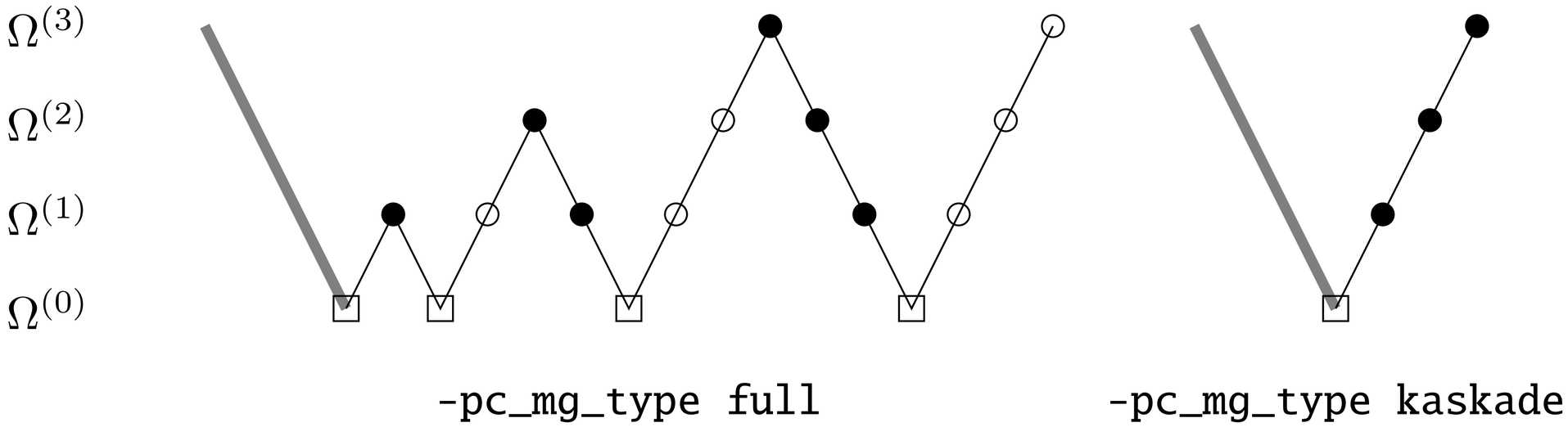


Figure 6.14. *One full (left) or Kaskade (right) cycle for a* `-da_refine 3 -pc_type mg` *run. The initial coarsening phase only restricts the residuals.*

FMG and Kaskade start at the Coarse level as an initial guess and refines

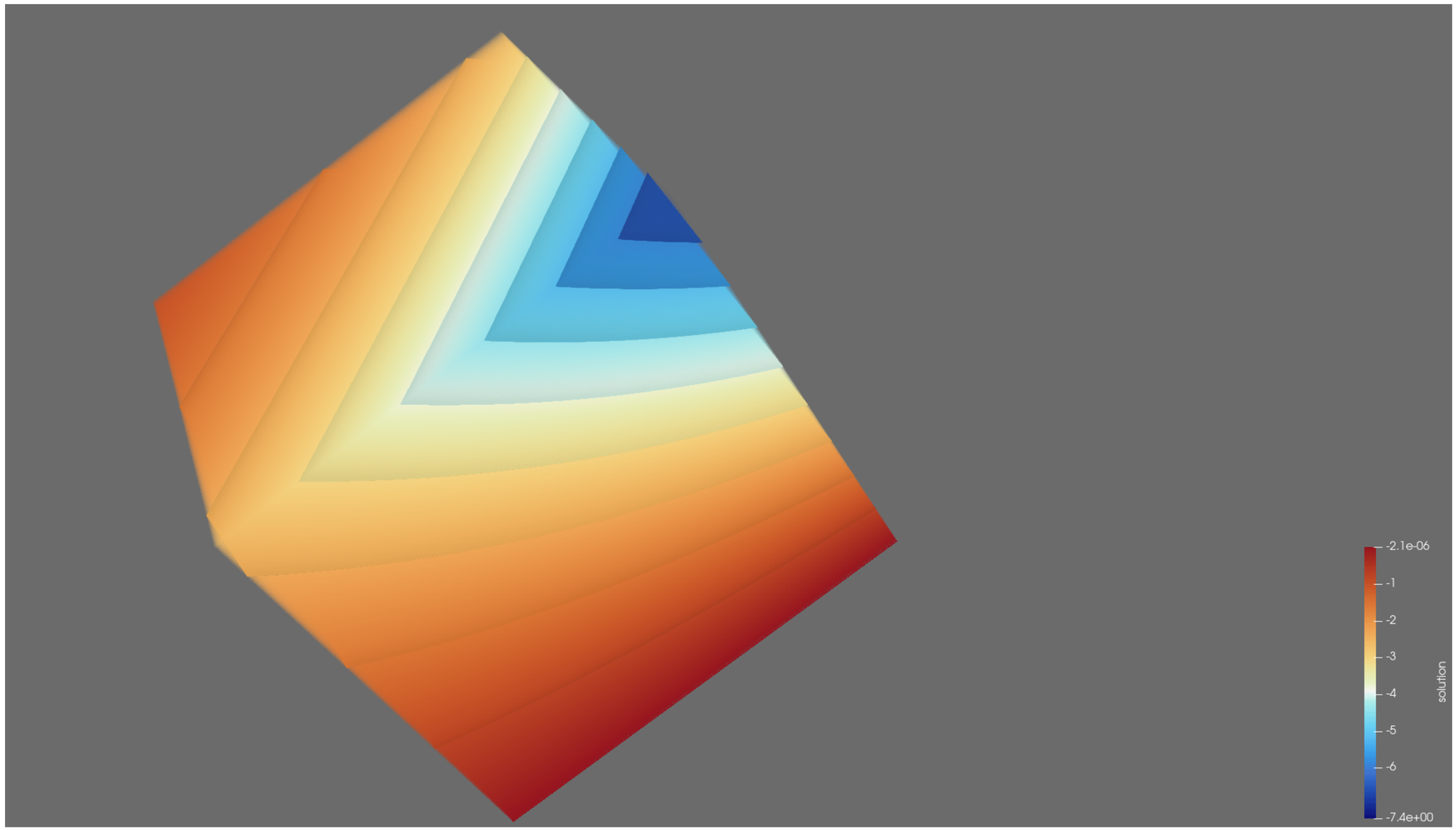
Time to mess around

p4pde's fish.c

- Solves $-\nabla \cdot C \nabla u = f$ in 1,2 and 3-D using 2nd order finite differences
  - $C = \mathrm{diag}(\mathbf{c})$ allows anisotropy in diffusion
- Writes the problem as a non-linear problem solves with SNES
- Includes two-manufactured solutions

Time to mess around

3-D solution with exp



Time to mess around

Effects of Anisotropy on solver choice

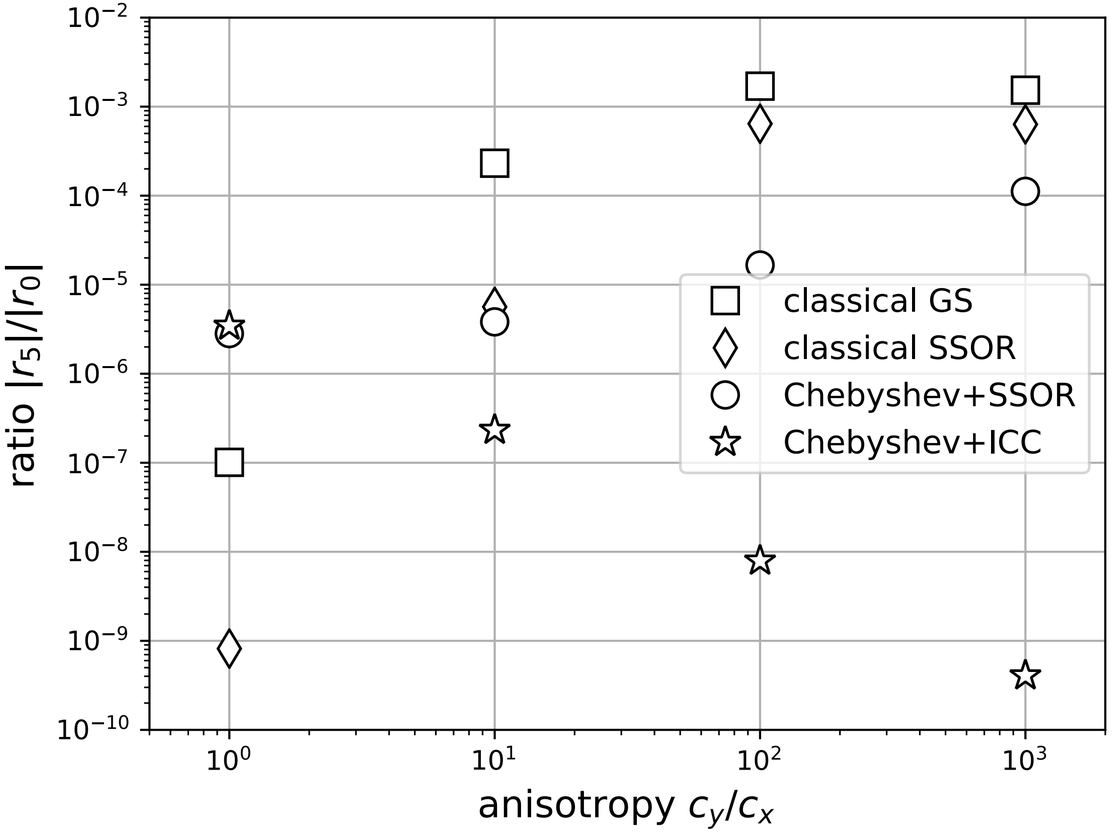
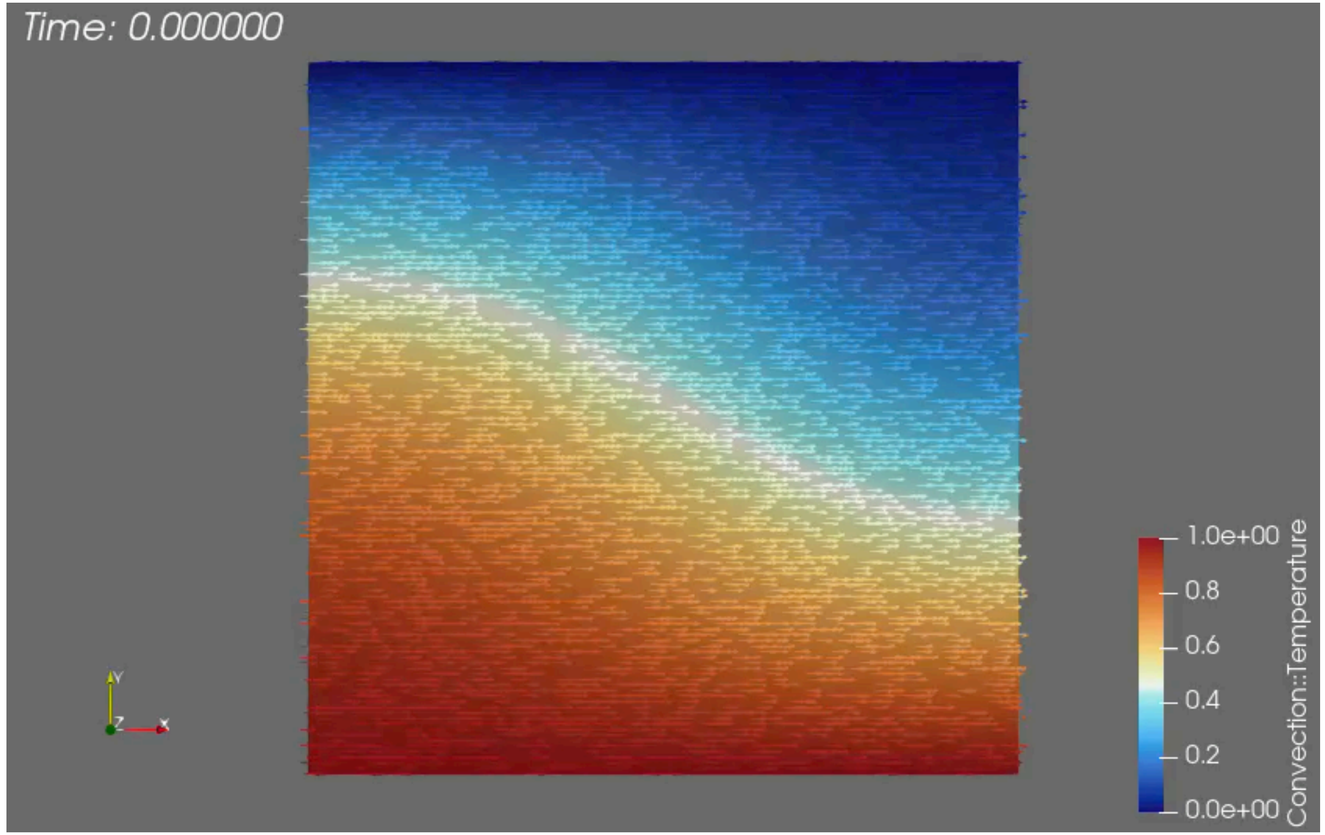


Figure 6.15. *When anisotropy c_y/c_x is strong in equation* (6.31)*, multigrid performance degrades, though Chebyshev smoothing can adapt. Our best smoother for strong anisotropy uses ICC preconditioning.*

Multi-physics block preconditioners

Towards effective multi-physics models



Advection-Diffusion Eq.

$$\frac{\partial T}{\partial t} + \mathbf{v} \cdot \nabla T = \frac{1}{Ra} \nabla^2 T$$

$$-\nabla \cdot \eta \left[(\nabla \mathbf{v}) + (\nabla \mathbf{v})^T \right] + \nabla P = T\hat{\mathbf{g}}$$

$$\nabla \cdot \mathbf{v} = 0$$

where $Ra = \dfrac{\rho_0 \alpha \Delta T h^3}{\eta \kappa}$ is the Raleigh #

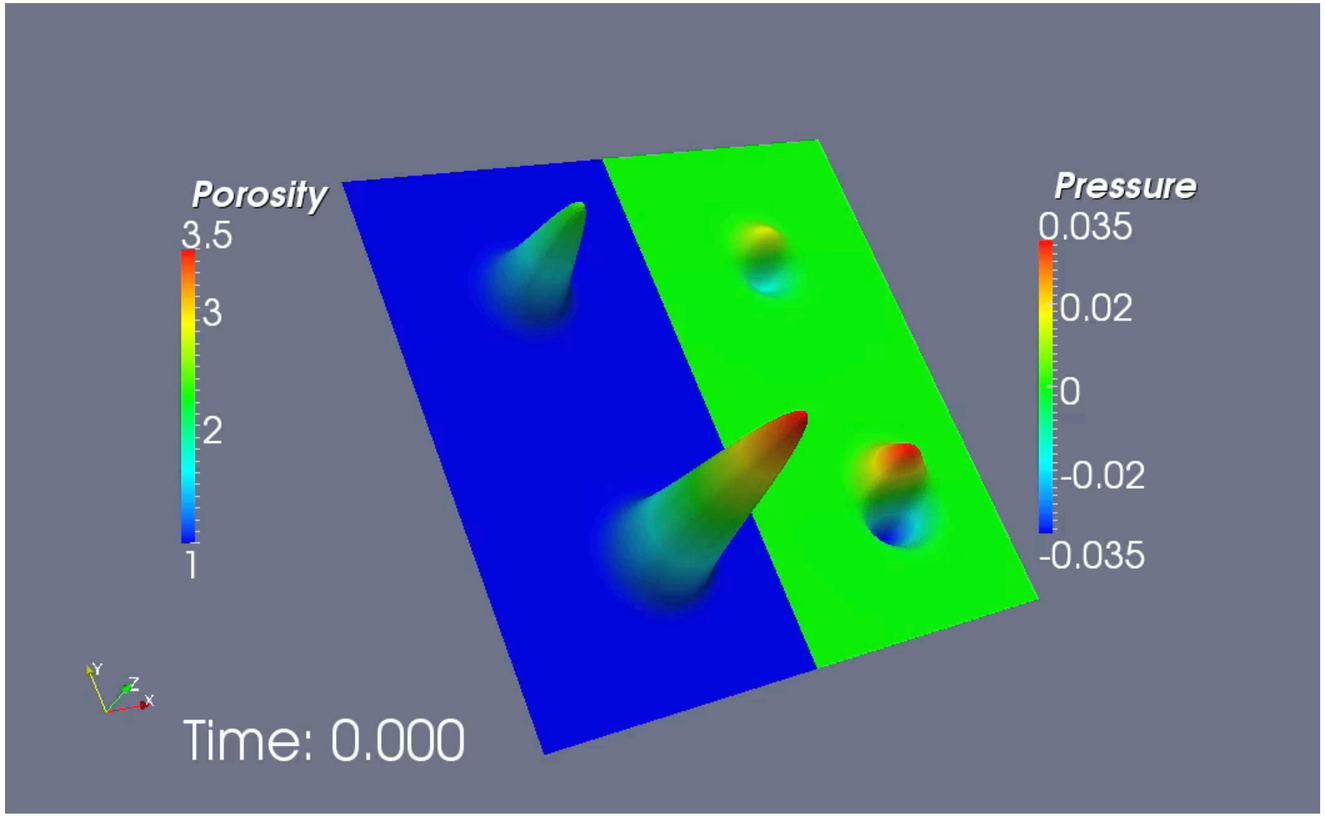
and $\eta = \eta(T, \mathbf{v})$ is the viscosity

Incompressible Stokes flow

Problem #1: Infinite Prandtl number thermal convection

Multi-physics block preconditioners

Towards effective multi-physics models



Porosity evolution

$$\frac{\partial \phi}{\partial t} + \mathbf{v} \cdot \nabla \phi = \alpha^2 \mathscr{P}$$

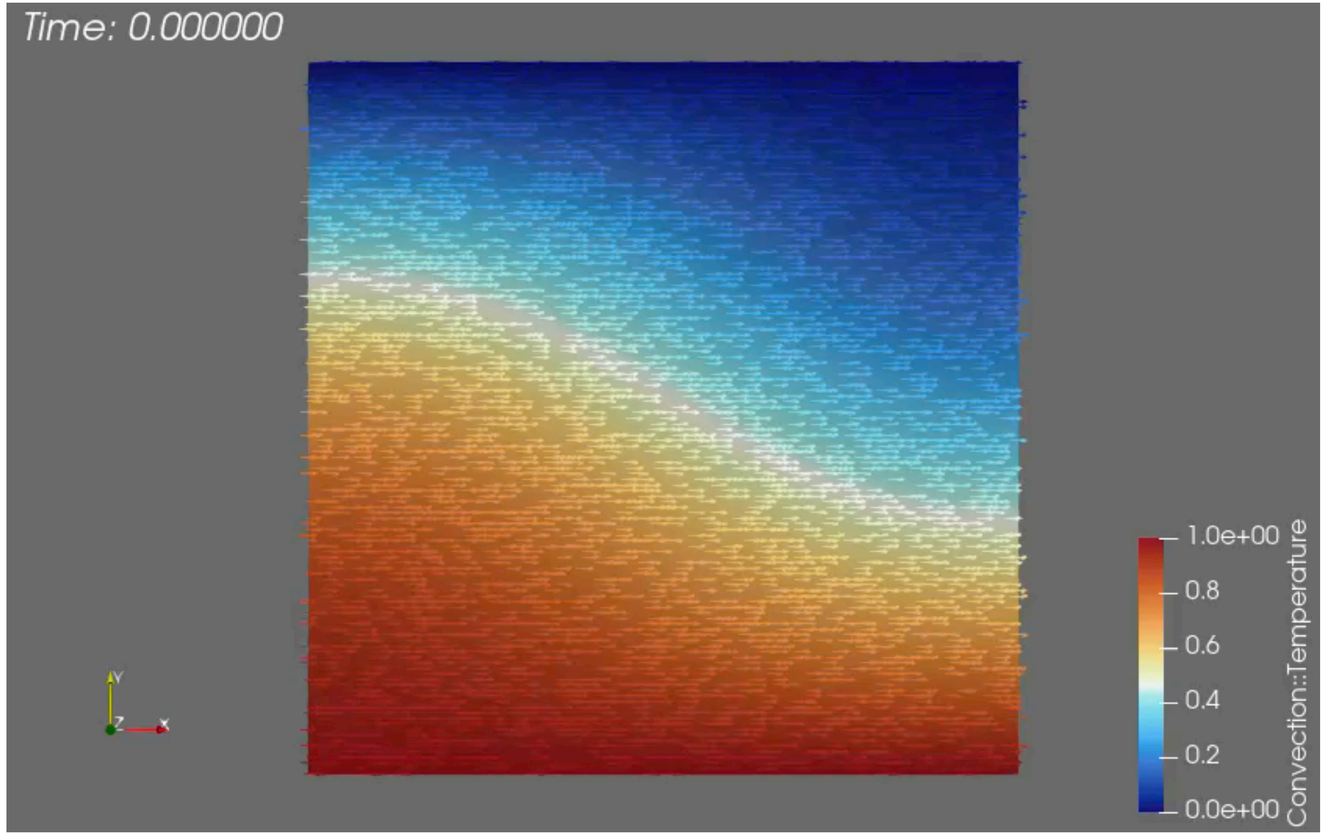
$$-\nabla \cdot K(\phi) \nabla \mathscr{P} + \alpha^2 \mathscr{P} = -\nabla \cdot K(\phi) \hat{\mathbf{g}}$$

where $\alpha = \frac{h}{\delta}$ is system height in "compaction lengths" and and $K(\phi) = \phi^n$ is the viscosity

Compaction pressure equation

Problem #2: Non-linear dispersive porosity waves

Thermal Convection



$$\frac{\partial T}{\partial t} + \mathbf{v} \cdot \nabla T = \frac{1}{Ra} \nabla^2 T$$

$$-\nabla \cdot \eta \left[(\nabla \mathbf{v}) + (\nabla \mathbf{v})^T \right] + \nabla P = T\hat{\mathbf{g}}$$

$$\nabla \cdot \mathbf{v} = 0$$

where $Ra = \dfrac{\rho_0 \alpha \Delta T h^3}{\eta \kappa}$ is the Raleigh #

and $\eta = \eta(T, \mathbf{v})$ is the viscosity

Problem #1: Infinite Prandtl number thermal convection

The Biharmonic Equation in split form

Assembly and storage

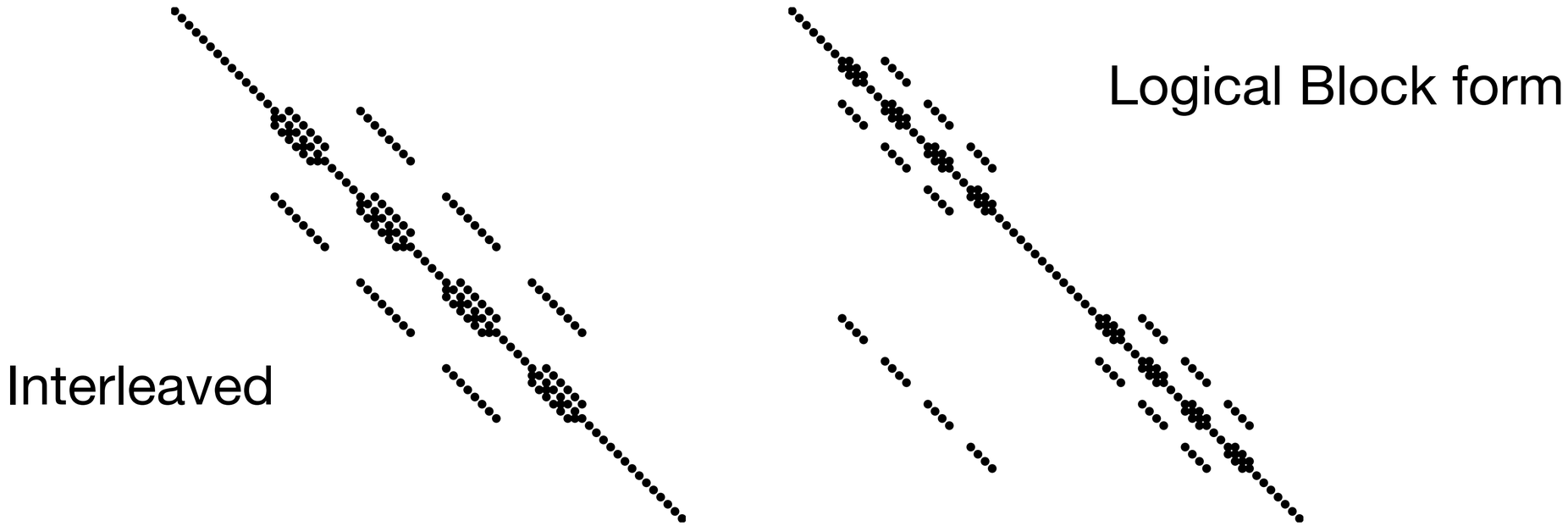


Figure 7.9. *Sparsity of T in the original, interleaved form (left) and reordered into blocks (right; equation* (7.11)*) with scalar Laplacians on the diagonal.*

Optimal solvers for $\nabla^2(\nabla^2 u) = f$

Monolithic vs block preconditioned mg

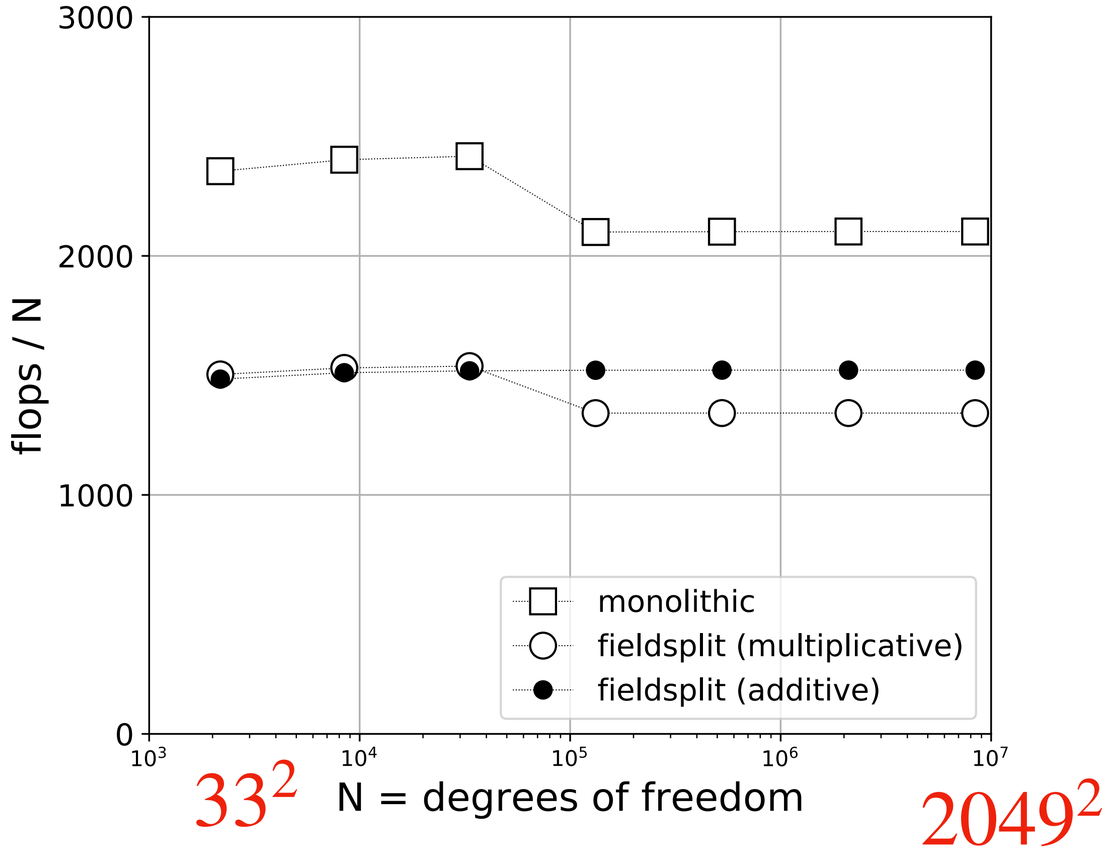


Figure 7.10. *Measured by work per degree of freedom, all three GMG solvers are optimal, with advantage to the* `fieldsplit` *methods.*

Example: PETSc Block Preconditioning & Solvers

$$J(u_i)\delta u = -r(u_i)$$

$$J = \begin{array}{c} \begin{matrix} v_x & v_y & P & T \end{matrix} \\ \begin{pmatrix} K_{11} & K_{12} & G_1 & C_1 \\ K_{21} & K_{22} & G_2 & C_2 \\ G_1^T & G_2^T & \mathbf{0} & \mathbf{0} \\ B_1 & B_2 & \mathbf{0} & A \end{pmatrix} \end{array}$$

Incompressible Stokes flow

$$-\nabla \cdot \eta \left[(\nabla \mathbf{v}) + (\nabla \mathbf{v})^T \right] + \nabla P = T\hat{\mathbf{g}}$$

$$\nabla \cdot \mathbf{v} = 0$$

$$\frac{\partial T}{\partial t} + \mathbf{v} \cdot \nabla T = \frac{1}{Ra} \nabla^2 T$$

Advection-Diffusion Eq.

Example: PETSc Block Preconditioning & Solvers

$$J(u_i)\delta u = -r(u_i)$$

$$J = \begin{pmatrix} K_{11} & K_{12} & G_1 & C_1 \\ K_{21} & K_{22} & G_2 & C_2 \\ G_1^T & G_2^T & \mathbf{0} & \mathbf{0} \\ B_1 & B_2 & \mathbf{0} & A \end{pmatrix}$$

Stokes Advection-Diffusion

Example: PETSc Block Preconditioning & Solvers

$$\tilde{J}_{\mathrm{PC}}(u_i)^{-1} J(u_i)\delta u = -\tilde{J}_{\mathrm{PC}}(u_i)^{-1} r(u_i)$$

$$J = \begin{pmatrix} K_{11} & K_{12} & G_1 & C_1 \\ K_{21} & K_{22} & G_2 & C_2 \\ G_1^T & G_2^T & \mathbf{0} & \mathbf{0} \\ B_1 & B_2 & \mathbf{0} & A \end{pmatrix}$$

$$J_{\mathrm{PC}} = \begin{pmatrix} K_{11} & K_{12} & G_1 & C_1 \\ K_{21} & K_{22} & G_2 & C_2 \\ G_1^T & G_2^T & M & \mathbf{0} \\ B_1 & B_2 & \mathbf{0} & A \end{pmatrix}$$

Example: PETSc Block Preconditioning & Solvers

$$\tilde{J}_{\mathrm{PC}}(u_i)^{-1} J(u_i)\delta u = -\tilde{J}_{\mathrm{PC}}(u_i)^{-1} r(u_i)$$

$$J = \begin{pmatrix} K_{11} & K_{12} & G_1 & C_1 \\ K_{21} & K_{22} & G_2 & C_2 \\ G_1^T & G_2^T & \mathbf{0} & \mathbf{0} \\ B_1 & B_2 & \mathbf{0} & A \end{pmatrix}$$

System
(fgmres, fieldsplit)

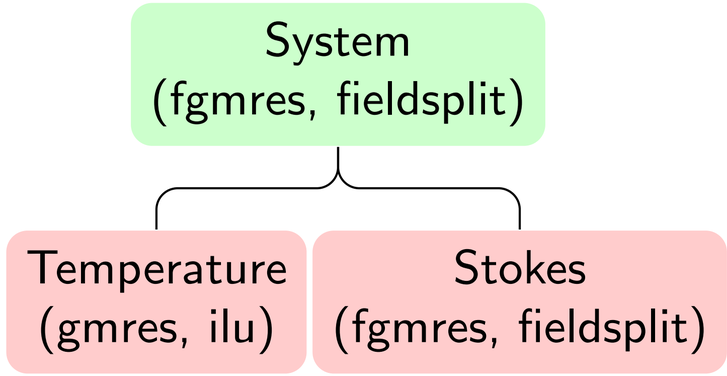
$$J_{\mathrm{PC}} = \begin{pmatrix} K_{11} & K_{12} & G_1 & C_1 \\ K_{21} & K_{22} & G_2 & C_2 \\ G_1^T & G_2^T & M & \mathbf{0} \\ B_1 & B_2 & \mathbf{0} & A \end{pmatrix}$$

Example: PETSc Block Preconditioning & Solvers

$$\tilde{J}_{\mathrm{PC}}(u_i)^{-1} J(u_i)\delta u = -\tilde{J}_{\mathrm{PC}}(u_i)^{-1} r(u_i)$$

$$J = \begin{pmatrix} K_{11} & K_{12} & G_1 & C_1 \\ K_{21} & K_{22} & G_2 & C_2 \\ G_1^T & G_2^T & \mathbf{0} & \mathbf{0} \\ B_1 & B_2 & \mathbf{0} & A \end{pmatrix}$$

$$J_{\mathrm{PC}} = \begin{pmatrix} K_{11} & K_{12} & G_1 & C_1 \\ K_{21} & K_{22} & G_2 & C_2 \\ G_1^T & G_2^T & M & \mathbf{0} \\ B_1 & B_2 & \mathbf{0} & A \end{pmatrix}$$

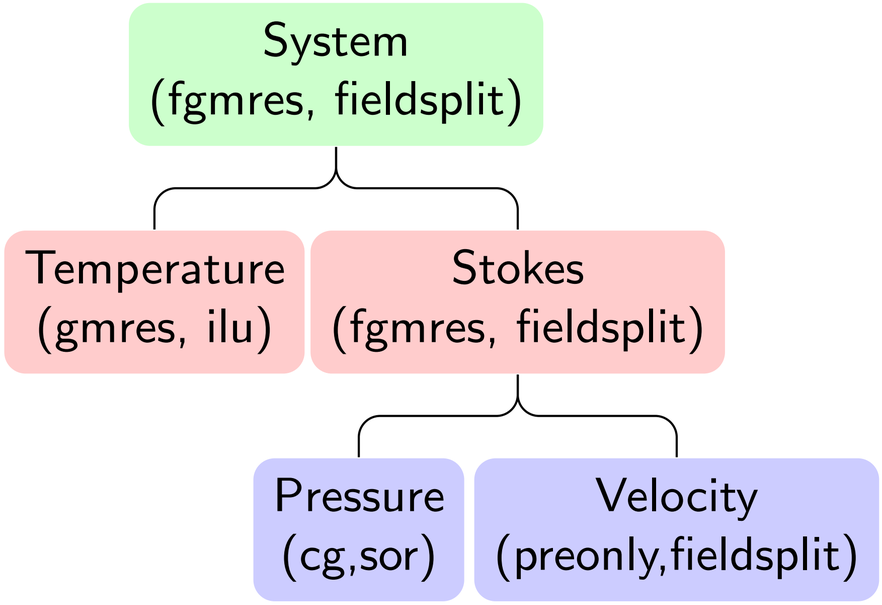


Example: PETSc Block Preconditioning & Solvers

$$\tilde{J}_{\mathrm{PC}}(u_i)^{-1} J(u_i)\delta u = -\tilde{J}_{\mathrm{PC}}(u_i)^{-1} r(u_i)$$

$$J = \begin{pmatrix} K_{11} & K_{12} & G_1 & C_1 \\ K_{21} & K_{22} & G_2 & C_2 \\ G_1^T & G_2^T & \mathbf{0} & \mathbf{0} \\ B_1 & B_2 & \mathbf{0} & A \end{pmatrix}$$

$$J_{\mathrm{PC}} = \begin{pmatrix} K_{11} & K_{12} & G_1 & C_1 \\ K_{21} & K_{22} & G_2 & C_2 \\ G_1^T & G_2^T & M & \mathbf{0} \\ B_1 & B_2 & \mathbf{0} & A \end{pmatrix}$$



Example: PETSc Block Preconditioning & Solvers

$$\tilde{J}_{\mathrm{PC}}(u_i)^{-1} J(u_i)\delta u = -\tilde{J}_{\mathrm{PC}}(u_i)^{-1} r(u_i)$$

$$J = \begin{pmatrix} K_{11} & K_{12} & G_1 & C_1 \\ K_{21} & K_{22} & G_2 & C_2 \\ G_1^T & G_2^T & \mathbf{0} & \mathbf{0} \\ B_1 & B_2 & \mathbf{0} & A \end{pmatrix}$$

$$J_{\mathrm{PC}} = \begin{pmatrix} K_{11} & K_{12} & G_1 & C_1 \\ K_{21} & K_{22} & G_2 & C_2 \\ G_1^T & G_2^T & M & \mathbf{0} \\ B_1 & B_2 & \mathbf{0} & A \end{pmatrix}$$